Kavosh One: approval plan

Kavosh AI and Robotics Academy Inc. | 3 November 2026 | Version 1.0

Decision summary

Build a complete academy operations platform with eight consistent workspaces: Overview, Academy, People, Learning, Finance, Reports, Community and Administration. Media is a shared library; approvals and notifications have one central inbox. Records link across workspaces without duplicate data entry.

The existing site is a React/TypeScript visual prototype running through Vinext on Cloudflare Workers, with a Drizzle/D1 schema foundation. Temporary screen state, example data and existing render checks do not establish production functionality. Preserve useful visual work and requirements; replace demonstration behaviour with real workflows.

Proposed production architecture: supported stable Next.js/React on Node, PostgreSQL, private R2 storage, a background worker and a documented REST API. Package the application for a VPS or managed container host. This changes the prototype's D1 and Worker-specific foundation; it is a proposed decision for approval, not an implemented migration.

Use a modular monolith: one maintained application with clear domain boundaries. Avoid microservices and Kubernetes initially. Separate worker processes protect interactive requests from exports, payroll runs, scanning and message delivery. Use maintained security components rather than inventing authentication or cryptography.

Design baseline: 5,000 student records plus guardians/staff, approximately 100 concurrent users, and measurable headroom. Capacity, uptime and security are acceptance targets, not guarantees. Test representative workloads and expand based on evidence.

Your external management bot connects directly through the same API and policy checks as the website. Configure its data scope, permitted actions, approval requirements and spending limits. It never needs browser automation or direct database credentials.

This plan preserves the full previously consolidated feature scope. Delivery stages organize the work; they do not remove requested features. Only planning artifacts were created for this review. No application implementation, deployment or service purchase is authorized by this document.

Approval boundary

Approval accepts the proposed product structure, architecture and staged implementation. Paid services, external messages, real payments, production imports and public launch require their applicable authorization. Vendor account entitlements and professional legal/accounting review remain specific launch gates.

1. Product scope and navigation

Overview and Academy

- Role-specific dashboard, exception cases, critical alerts, approvals, upcoming sessions and drilldowns.
- Multiple isolated academies; platform owner creates each academy and its one non-transferable master account. Platform access is audited. Kavosh subscription is free.
- Country, province/region, city, hub, class and session hierarchy; hub-specific names, logos, contacts, maps, currencies and time zones.
- Editable courses/modules; generated class codes; effective-dated teachers, TAs, volunteers and student enrollment.
- Weekly and irregular calendars, holidays, exceptions, online links, cancellation/reschedule approvals, year pathways and competitions with flags/links.

People and Learning

- Student and guardian records, relationships, legal/preferred names, DOB, contacts/WeChat, shirt size, restricted care data, recruiter and joining/leaving dates.
- Admin-assisted registration or secure self-service invitations; email/phone verification; BoldSign enrollment package and consent history.
- Session attendance, individual staff time, shared rich reports/media, private family reports, six 1-5 learning ratings, replies and approval/override cases.
- Homework by class/student, submissions, solutions, grades and controlled publication. Parents and students receive different views.
- Salaried/non-teaching staff, teachers, contractors and configurable employment/payroll settings.

Finance and Reports

- Double-entry accounts/journals; tuition/installments; online/offline payments; receipts; tax; expenses; refunds; credits; loans; reimbursements; currency conversion; bank import/reconciliation.
- Recruiter allocations and collector-aware hub settlements; recurring bills; salaried/hourly/project compensation; paystubs; T4/T4A exports without CRA filing.
- Inventory with individual/bulk quantities, purchase records, pictures, locations, custodians and disposal history.
- Permission-aware financial, learning, attendance, staffing, inventory and operational reports; filters, charts, drilldowns, saved presets, PDF/Excel/CSV and scheduled delivery.
- Cash, accrual and forecast views, including tuition due dates, rent, salaries and scheduled teaching estimates.

Community and Administration

- Student/teacher/hub profiles, QR links, achievements, certificates, pinned academy history, posts/stories, verified-user comments, moderation and weekly leaderboards. No direct private messages.
- Custom roles/presets/scopes; notifications; rule builder; audit diff/history; media browser/search; integrations; bot controls; branding; localization.
- Email/SMS/portal reminders, polls and record-based family/staff discussions. Parent class reminder initially 24 hours; teacher reminder initially one hour.
- English, French, Simplified Chinese; tenant/hub branding; subscription plans, read-only grace/cancellation states and confirmed tenant deletion.

2. Architecture and data integrity

Request and processing flow

Browser or bot -> protected edge/reverse proxy -> authenticated API -> authorization and domain service -> PostgreSQL transaction -> transactional outbox -> worker -> provider. Files use a separate private storage path. Signed provider events return through verified webhook endpoints.

Application structure

- One repository and shared types, with modules for identity, academy, people, scheduling, learning, accounting, payroll, files, community, automation and reporting.
- Standard supported Next.js/Node deployment in reproducible containers; exact supported versions rechecked at implementation and release. Retain React/TypeScript and useful existing components.
- PostgreSQL is the authoritative business store. Continue Drizzle only where it fits the PostgreSQL schema. Use relational tables for financial and permission invariants; reserve JSON for validated flexible settings and document content.
- PostgreSQL-backed durable jobs/outbox initially, using a maintained implementation. Add Redis or separate analytics infrastructure only when measurements justify it.
- R2 stores media, documents and exports privately; PostgreSQL stores searchable metadata and permission links. Separate public derivatives from private originals.

Tenant and permission model

Every business record has an academy boundary. Composite keys/constraints prevent cross-academy references. Application checks enforce feature rights plus record scope; database row policies provide a second boundary. Runtime database roles must not own tables or bypass row security. Transaction-local tenant context must be reset safely with pooling. Platform-owner access uses a separate audited privileged path, not an unrestricted routine connection. [S4]

Roles can combine grants, but explicit denials and narrower protected-field restrictions take precedence. A user cannot grant rights outside their delegation. Scope cannot expand through exports, search, files, aggregate charts, notifications or bot calls. Guardians' access follows verified relationships and relevant dates.

Reliability and financial correctness

- Atomic writes; constraints; request idempotency; optimistic locking for concurrent edits; UTC timestamps plus original time zone and date-only business fields.
- Exact monetary values with currency-specific minor units and decimal rates. Never floating-point arithmetic for posted amounts. Document rounding/residual allocation.
- Balanced journal header/lines; immutable posted records; correcting reversals; closing periods; evidence and rule versions retained.
- Write business change, audit event and outbox event atomically. Separate secured audit copies make tampering detectable; no claim that a database owner is technically unable to alter a local table.
- Worker retries are at-least-once; unique effect keys and provider reconciliation prevent duplicate charges, sends and ledger postings.
- Private pages are never cached publicly. Cache keys include tenant, scope and permission version when private caching is justified.

3. Management bot and automation

Easy direct connection

Provide a versioned /api/v1 REST API, OpenAPI specification, example requests and a small connection guide. An optional MCP adapter can expose approved API actions to compatible agents; REST remains the core contract. No custom model vendor is required to use the API.

An administrator registers a bot, chooses its academy/hubs and permitted operations, then securely issues expiring machine credentials. Prefer standard OAuth machine credentials where supported. An initial scoped opaque-token option must use hashed storage, expiry, rotation and revocation. Never reuse the master password or give the bot master identity.

API capabilities include querying records, creating draft entries, maintaining schedules, preparing reports, proposing settings changes, requesting approvals, sending allowed notifications and taking authorized management actions. Access to payroll, health data and exports requires separate scopes.

Configurable authority

- Observe: read permitted records, detect issues and summarize.
- Propose: prepare a change and explain impact, without applying it.
- Act with approval: apply the exact version/payload approved by a human.
- Act independently: only allowed actions within configured scope, amount, audience and frequency limits.

Configure those modes per action. Suggested initial preset: read operations, prepare reports, draft routine corrections; approval for money movement, public student content, bulk messaging, role changes, retention changes and deletions. These are defaults you can adjust within mandatory platform safeguards.

Control and accountability

Each run records bot identity, initiator/delegation, request ID, evidence used, proposed changes, approval, executed outcome and provider result. Explain results with links to source records. Approved data changing before execution invalidates the approval. The bot cannot approve its own request by switching identities or spawning another run. Human master self-approval does not automatically extend to machine credentials.

Include preview/dry-run, amount and message budgets, maximum batch sizes, rate limits, pause switch, revocation, notifications, retry control and compensating actions where feasible. Never promise rollback of an email already sent or a completed external payment.

Treat reports, uploads, comments and external messages as untrusted data, not instructions to the bot. Tool allowlists, destination restrictions, minimal data disclosure, isolation between academy runs and prompt-injection tests are mandatory. Model output must pass ordinary API validation. Do not let an agent execute arbitrary SQL, shell code or unrestricted rule scripts. [S3]

Shared automation system

Deterministic rules handle reminders, thresholds and escalations cheaply. Use a model only where interpretation adds value. Rules use registered triggers/conditions/actions, effective dates, simulation, versioning, approvals, delivery history and loop prevention. Every requested operational area must be represented in the trigger/action catalogue and acceptance register.

4. Finance, payroll and reporting controls

Accounting scope

Include chart of accounts, general ledger, trial balance, profit/loss, balance sheet, cash-flow views, receivables/payables, account balances, reconciliation, tax summaries, period controls and supporting documents. Cost dimensions support student, class, hub, geography and academy reporting without duplicating ledger entries.

Every payment records amount/currency/date, collector, ultimate beneficiary, method, account, reference, receipt and tax treatment. Allocation rules use actual paid installments and record who holds the funds. Loans, opening balances and inter-hub transfers must not be mistaken for income. Online payment intent, settlement, fee, refund and chargeback each have a defined accounting treatment.

Non-master submissions require upper-level approval. Posted corrections use reversal/adjustment workflows. Manual bonuses and unusual entries create critical review cases. A UI label of 'paid' must correspond to a payment record; approval and bank reconciliation remain separate statuses.

Compensation rules

Salary, hourly, tuition percentage, headcount thresholds/increments and project pay are effective-dated. Define eligible headcount from joining/leaving dates and the rule's documented basis. Overlapping hourly intervals pay once at the highest applicable rate; preserve original intervals and show attribution. Percentage/project components need explicit rules and must not be accidentally removed by hourly overlap logic.

Each staff member logs time. One shared attendance/report/media set may satisfy the session. Missing evidence blocks the affected session's internal approval, subject to a documented override. Round to nearest five minutes while preserving actual time and the rounding delta; use a configurable five-minute excess tolerance initially.

Employment law takes precedence over configuration: an employee's earned wages cannot simply disappear because media/report evidence is missing. Escalate unresolved cases before payroll due dates, retain gross earned liability and permit documented administrative resolution. Test rounding and overlap treatment for lawful employee pay. Employee/contractor classification is substantive, not chosen merely by selecting T4/T4A. Accountant review is required before live payroll.

Tax, assets and forecasts

Tax treatment is explicit and independent of cash/card/e-transfer. Non-taxable choices need a valid classification; all transactions remain reportable. Use current effective-dated CRA/payroll rules, validated examples and annual updates. No CRA filing is included.

Record tool purchases as requested expenses in management reporting and preserve their history after disposal. Show usable inventory at purchase cost in a separate management view. Formal balance-sheet treatment must follow the approved accounting policy; do not count an expensed purchase again as a formal asset. No depreciation engine is included.

Forecasts separate actual/committed/estimated values and show source, exchange rate, assumptions and update time. Reports reconcile to journal lines and approved source records. Saved layouts, filters and schedules cannot bypass data scope. Large exports run as jobs with expiring download links.

5. BoldSign and service integration

Confirmed BoldSign path

Retain BoldSign as the signing provider. Templates are versioned in the academy; requests reference the student/enrollment and guardian. The service creates a signing request, delivers a secure or embedded signing flow, tracks status, retrieves the completed document and audit evidence, and updates eligibility only after verified completion.

BoldSign documents HMAC-SHA256 webhook signatures. Validate the raw body, timestamp and signature, reject replay/duplicates, and reconcile ambiguous/out-of-order events against provider status. Never activate enrollment from a browser success redirect alone. Keep provider document IDs, consent choices and archived completed files. [S5]

The API supports email OTP or SMS OTP signer authentication. Website verification of email and phone is separate evidence; do not claim BoldSign independently proves both factors simultaneously without testing the precise flow and audit trail. No provider can guarantee every waiver will be enforceable in every Canadian court. Ontario electronic-signature law supports the approach, but document content, evidence and exceptions still matter. [S6, S7]

Verified published pricing

BoldSign Enterprise API starts at USD 30/month with 40 envelopes; additional envelopes are USD 0.75 each. A normal web-app subscription does not establish entitlement to live API sending. Free API sandbox documents are test-only. SMS authentication has a published USD 0.20/message starting charge, with country-dependent pricing and retries potentially adding cost. Confirm the account region, plan, country coverage and invoice terms before activation. [S6, S8]

These are published plan facts, not a purchase or an assessment of the user's current account. At 5,000 enrollments, envelope and verification costs must be modeled separately from hosting; not every registered student necessarily signs every month.

Other integrations and procurement gates

- Payments: hosted/tokenized collection, required Canadian credit/debit methods, refunds, disputes, signed webhooks and reconciliation. Card debit availability must be verified by card/network; do not imply Interac acceptance without support.
- Email/SMS: delivery quality, Canadian and international coverage, authentication, opt-out, signed callbacks, support and spend limits. SMS is not assumed receiver-paid.
- Authentication: maintained provider/library supporting verified email/phone, recovery, device trust and machine access. Do not invent cryptography.
- Banking: manual entry, CSV import and balance-only snapshots first; future read-only connectors depend on actual bank coverage and credentials.
- Media: R2 primary; authorized Google Drive export/backup. Backup independence and restore testing matter beyond copying within one account.

Before committing to a paid provider, produce a short comparison using current official capabilities/prices plus relevant independent user feedback, integration testing and essential limitations. No new paid vendor is selected here. Plugin discovery found useful design/database tools but did not establish a connected BoldSign account. A plugin is optional development assistance, not a production integration dependency.

6. Security acceptance checklist

Use OWASP ASVS 5.0 Level 2 as the minimum verification baseline, plus selected stronger controls for privileged accounts/financial actions, API security risks and agent-specific threats. These are engineering standards; passing a checklist is not a promise of invulnerability. Every item below needs an implementation owner and test evidence before release. [S1-S3]

Identity, browser and API

- Verified email/phone; short-lived single-use invitation/reset/verification tokens; OTP expiry, attempt caps and resend/cost limits; account-enumeration resistance.
- Trusted-device expiry/revocation for the master; new-device verification, recovery notifications, session rotation, idle/absolute expiry and emergency revocation. Recommend stronger passkey/TOTP options alongside the agreed SMS flow.
- Secure/HttpOnly/scoped session cookies; suitable SameSite; CSRF tokens/origin checks for cookie-authenticated changes; strict CORS; canonical host validation.
- CSP, clickjacking protection, nosniff, referrer/permissions policies, safe redirects and no long-lived browser-storage tokens. Permit only explicitly required signing/payment embed origins.
- Runtime input schemas, length/type/range limits, parameterized SQL, output escaping and rich-text sanitization. Test SQL injection, XSS, CSRF, SSRF, path traversal, mass assignment and object/property authorization.
- Idempotency and concurrency protection for money and approvals; API version inventory, scoped machine access, signature verification and replay protection for webhooks.

Infrastructure, files and database

- Edge DDoS protection plus WAF/rate limits appropriate to the purchased plan; origin access restricted so attackers cannot bypass the edge. Verify rules rather than assuming all Cloudflare plans include the same controls.
- Private database networking, TLS, encryption at rest, least-privilege application/migration/backup accounts, pooling limits, query timeouts and tenant-isolation tests.
- Upload quarantine, content/type verification, size/duration/count caps, malware scanning, safe image transforms, controlled previews, metadata/location stripping from public derivatives and unguessable storage keys.
- Short-lived signed file access with fresh authorization, no public private bucket, no credentials in frontend/logs, allowlisted outbound destinations and safe PDF rendering without arbitrary network access.
- Secrets manager/injected secrets, rotation, audit access, production/staging separation, synthetic test data, non-root containers, host patching and network least privilege.

Operations and assurance

- Dependency/advisory/secret scans, reproducible lockfile installs, reviewed migrations, software inventory/license checks and minimal third-party scripts.
- Bounded jobs/exports/AI/SMS spending, fair tenant quotas, bot abuse controls, queue backpressure, dead-letter handling and service degradation procedures.
- Encrypted backups, separate credentials/copy, legal holds, deletion propagation, restore drills and documented incident response.
- Structured redacted logs, abnormal access alerts, audit integrity, vulnerability reporting contact, independent penetration testing and retest of findings.

7. Privacy, legal and operational policies

Ontario/Canada baseline, international extensions

Create a versioned privacy notice, terms of service, enrollment agreement/waiver, public-media choices, acceptable-use/community rules, moderation/takedown process, accessibility statement, cookie notice/preferences, cancellation/refund policy, copyright/IP permissions, security contact and complaints process. Prepare tenant data-processing terms, subprocessor register, retention schedule and incident plan. Legal text must describe actual behaviour, not generic promises.

PIPEDA planning includes purpose limitation, minimal collection, safeguards, access/correction, consent records, accountable privacy contact and breach assessment/notification where required. Maintain breach records; recordkeeping duration is set from the applicable rule. Separate legally necessary processing from optional public profiles, marketing and analytics. [S9]

Children require guardian-managed onboarding and age-appropriate notices. Default profiles private; public visibility needs documented choice and guardian authorization where applicable. Public posts must not reveal DOB, health data, exact class schedules or private reports. Student-selected visibility cannot override guardian/legal restrictions.

The requirement that every user verify a phone conflicts with younger students who lack phones. Proposed resolution: a student account can use a verified linked guardian contact, with guardian-controlled recovery, rather than forcing a child's own phone or allowing shared-number account takeover. Keep student login identity separate. This is a proposed refinement for approval.

CASL: classify message purpose; track consent/source; identify sender; honor unsubscribe/suppression for commercial email/SMS. Do not evade consent by labeling promotion operational. Essential service notices must contain only appropriate service content. [S10]

Design to WCAG 2.2 AA, with keyboard/screen-reader testing and accessible alternatives for charts and PDFs. Ontario public-website obligations depend on organization type/size; do not claim the statutory threshold is automatically met. [S11]

International availability requires a jurisdiction register. Assess Quebec/other provincial requirements and EU GDPR, US COPPA/state rules and each relevant Asian market before enabling affected operations. GDPR can apply to non-EU companies offering services to people in the EU; COPPA amendments changed requirements relevant in 2026. Do not call one Ontario consent form worldwide compliance. [S12, S13]

Retention and deletion conflicts resolved

Ninety-day backups are disaster recovery, not business-record retention. Financial records generally need at least six years from the relevant tax-year end, with exceptions and legal holds. A tenant's 30-day cancellation/deletion flow must preserve records lawfully required and explain them. [S14]

No arbitrary media expiry is proposed. However, indefinite child-data retention cannot be promised where law requires purpose-based deletion. Provide purpose/consent review, withdrawal, legal-hold exceptions and controlled deletion; restore procedures reapply deletion records. Backups expire by policy; avoid claiming immediate removal from immutable backups.

8. Professional UX and efficient operation

Design direction

Use a bespoke CRM design system grounded in Kavosh's unchanged red/black logo: neutral surfaces, precise typography, consistent spacing, clear hierarchy and purposeful red accents. Design light/dark themes without turning finance screens into neon control panels. Other academies/hubs can supply their branding within contrast-safe tokens.

UI/UX skill research supports clear flat surfaces, keyboard navigation and breadcrumbs. Its generic landing-page/palette suggestions do not fit this product and are rejected. The landing-page design skill explicitly excludes dense dashboards; it will apply only to public marketing surfaces. Select one accessible component foundation after a compatibility proof, not several overlapping UI systems.

Use stable navigation, global scope/search, saved views, dense readable tables, filter chips, contextual record drawers and progressive disclosure for advanced settings. Mobile uses task-focused layouts and accessible table/card alternatives without removing financial detail. Every visible control has loading, empty, error, permission-denied and success states.

Charts answer real questions: income/expense trends, overdue aging, attendance patterns, class capacity and learner development. Include labels, units, source period, accessible tables and drilldowns. Do not replace exact accounting numbers with decorative graphics.

Animations communicate navigation, expansion, progress and state changes, normally about 150-250 ms. Use transform/opacity, honor reduced motion, pause offscreen media and avoid perpetual effects or forced scroll choreography in work screens.

Resource and resilience design

- Paginate/filter at the server; index demonstrated query patterns; avoid N+1 queries; bound date ranges and query time; inspect query plans.
- Load charts/editors only where needed; optimize image variants; direct controlled uploads avoid proxying large media through the application; process video outside requests.
- Aggregate expensive analytics asynchronously with freshness timestamps. Use a reporting replica only when measured pressure warrants it.
- Durable queues, bounded concurrency, retries with backoff/jitter, deduplication, provider timeouts and circuit breakers. No synchronous bulk SMS/PDF/payroll work in page requests.
- Stateless app instances behind health-aware load balancing; safe rolling deployments, connection pooling and graceful shutdown. A single VPS is a deployment option, not high availability. A production uptime design needs failure-domain redundancy and database recovery/failover.
- Shared invalidation for multi-instance caches, tenant-safe keys and cache isolation. Never trade privacy for faster caching. [S15]

Proposed measurable targets

At representative data volume: p95 ordinary API response below 500 ms excluding external providers; LCP below 2.5 s, INP below 200 ms and CLS below 0.1 at the 75th percentile. Establish a 100-user workload, burst tests around 300 users, export/payroll contention and sustained-load tests. Record throughput, errors, CPU/memory, database pressure and monthly cost. These are proposed test budgets, not observed results.

9. Delivery and verification plan

Stage 1: specification and design proof

Map all requirements to screens, APIs and tests. Resolve workflows, permissions and financial examples. Review desktop/mobile designs for dashboard, student, session, finance and automation. Validate runtime/auth/UI choices in a disposable proof.

Stage 2: secure foundation

Implement PostgreSQL, roles/scopes, verification/recovery, audit, secure files and the bot API. Demonstrate persistent records, cross-tenant denial, field restrictions, secure invites and credential revocation.

Stage 3: daily operations and BoldSign

Implement enrollment/signing, schedules, attendance/time, reports/media, homework and contextual communication. Test joining dates, daylight saving, guardian access, missing evidence, overrides and duplicate/failed signing events.

Stage 4: financial core and payroll

Implement accounts, tuition/payments, shares/settlements, expenses, banking and payroll. Verify journal balance, refunds, partial installments, tax, currencies, overlap/rounding, bonuses, reversals and closed periods. Accountant-reviewed examples are required before live use.

Stage 5: reports, automation and management agent

Implement reporting, forecasts, exports and rules. Connect the user's bot and verify scope, previews, approvals, budgets, prompt-injection resistance and stop controls. Compatibility depends on the bot client's API/MCP support.

Stage 6: community, multi-school administration and launch

Implement profiles, moderation, achievements, QR links, subscriptions and retention. Verify security/accessibility, workload and restore tests; stage rollout. Connect approved domains and verify DNS, certificates and origin protection.

Definition of complete

Every requested feature has a traceable acceptance result. Tests use real stored state and reloads; success toasts are not evidence. Verify adverse cases: forbidden access, validation errors, concurrent edits, provider outage, duplicate jobs and interrupted uploads. Separate sandbox and live-provider evidence.

Capture browser evidence at approximately 360/390 px mobile, 768 px tablet and 1280/1440/1920 px desktop, plus supported real browser/device checks. Review clipping, alignment, text, contrast, touch, focus and motion. Fix and retest affected cases. Do not claim a perfect score, zero risk or untested device compatibility.

Each milestone report states implemented, tested, failed, deferred and blocked. Release gates include no unresolved critical security/financial defects, reconciled financial examples, a successful restore, load results and named operational ownership.

Proposed recovery defaults and remaining gates

Keep daily encrypted database backups for 90 days around 3 AM academy time, while staggering tenant jobs. Add continuous recovery where the chosen host supports it. Proposed recovery-point target: 15 minutes; recovery-time target: four hours. Verify feasibility and price before committing. Independent copies, file recovery and quarterly restore drills are included in the operational design.

No questionnaire is needed now. Remaining evidence gates are the existing BoldSign plan/region/template IDs, current vendor budget/quotes, final legal/accounting approval and deployment credentials. Ask one focused question only when that decision is necessary. Implementation waits for approval of this plan.

10. Source register

Official sources checked on 3 November 2026. Product decisions in this plan are engineering judgments, not claims that the referenced organizations certify Kavosh. No independent provider comparison or live integration test was performed in this planning pass.

- S1. OWASP ASVS 5.0: https://asvs.owasp.org/
- S2. OWASP API Security Top 10: https://api-security.owasp.org/editions/2023/en/0x11-t10/
- S3. OWASP agentic application risks for 2026: https://genai.owasp.org/2025/12/09/owasp-genai-security-project-releases-top-10-risks-and-mitigations-for-agentic-ai-security/
- S4. PostgreSQL row security and bypass roles: https://www.postgresql.org/docs/17/ddl-rowsecurity.html
- S5. BoldSign webhook verification: https://developers.boldsign.com/webhooks/verify-webhook-events/
- S6. BoldSign API plans: https://boldsign.com/electronic-signature-pricing/?plan=webapp
- S7. Ontario Electronic Commerce Act: https://www.ontario.ca/laws/statute/00e17
- S8. BoldSign usage/add-on charges: https://support.boldsign.com/kb/article/21241/pay-as-you-go-for-boldsign
- S9. OPC breach obligations: https://www.priv.gc.ca/en/privacy-topics/privacy-for-businesses/privacy-breaches-at-your-business/gd_pb_201810/
- S10. CRTC CASL guidance: https://crtc.gc.ca/eng/com500/faq500.htm
- S11. W3C WCAG 2.2: https://www.w3.org/TR/WCAG22/ ; Ontario applicability: https://www.ontario.ca/page/when-public-websites-are-not-accessible
- S12. European Commission GDPR applicability: https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/application-gdpr_en
- S13. FTC COPPA current guidance: https://www.ftc.gov/business-guidance/resources/complying-coppa-frequently-asked-questions
- S14. CRA business record retention: https://www.canada.ca/en/revenue-agency/services/tax/businesses/topics/keeping-records/where-keep-your-records-long-request-permission-destroy-them-early.html
- S15. Next.js self-hosting guidance: https://nextjs.org/docs/app/guides/self-hosting

How skills influenced this plan

Ponytail full mode: modular monolith, reuse existing code where suitable, no speculative microservices, preserve financial/security validation. UI/UX Pro Max: task-based navigation, responsive tables, useful charts and accessibility; generic palette/landing recommendations rejected. Design Taste Frontend: public-facing style guidance only, because its instructions explicitly exclude dense dashboards. Security Best Practices: browser/server boundaries, cookies, validation, tenant authorization, safe files and webhook controls. Plugin Management: checked integration tooling without installing or connecting accounts. PDF skill: render and visually verify this review document.